

Regression & Its Evaluation | Assignment

Question 1: What is Simple Linear Regression?

Answer:

Simple Linear Regression is a statistical method used to model the relationship between one independent variable (X) and one dependent variable (Y) using a straight line. The equation is:

$$Y = \beta_0 + \beta_1 X + \epsilon$$

where β_0 is intercept, β_1 is slope, and ϵ is error.

Question 2: What are the key assumptions of Simple Linear Regression?

Answer:

- ☐ Linearity – Relationship between X and Y is linear
- ☐ Independence – Observations are independent
- ☐ Homoscedasticity – Constant variance of errors
- ☐ Normality – Errors are normally distributed
- ☐ No multicollinearity (for multiple regression)

Question 3: What is heteroscedasticity, and why is it important to address in regression models?

Answer:

Heteroscedasticity occurs when the variance of residuals is not constant across all levels of the independent variable.

It is important because it leads to inefficient estimates and unreliable hypothesis testing.

Question 4: What is Multiple Linear Regression?

Answer:

Multiple Linear Regression models the relationship between one dependent variable and two or more independent variables.

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \epsilon$$

Question 5: What is polynomial regression, and how does it differ from linear regression?

Answer:

Polynomial Regression models nonlinear relationships by adding powers of X.

Example:

$$Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \epsilon$$

Difference: Linear regression fits straight line, polynomial fits curve.

Question 6: Implement a Python program to fit a Simple Linear Regression model to the following sample data:

- $X = [1, 2, 3, 4, 5]$
- $Y = [2.1, 4.3, 6.1, 7.9, 10.2]$

Plot the regression line over the data points.

Answer:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
```

```
X = np.array([1,2,3,4,5]).reshape(-1,1)
Y = np.array([2.1,4.3,6.1,7.9,10.2])
```

```
model = LinearRegression()
model.fit(X, Y)
```

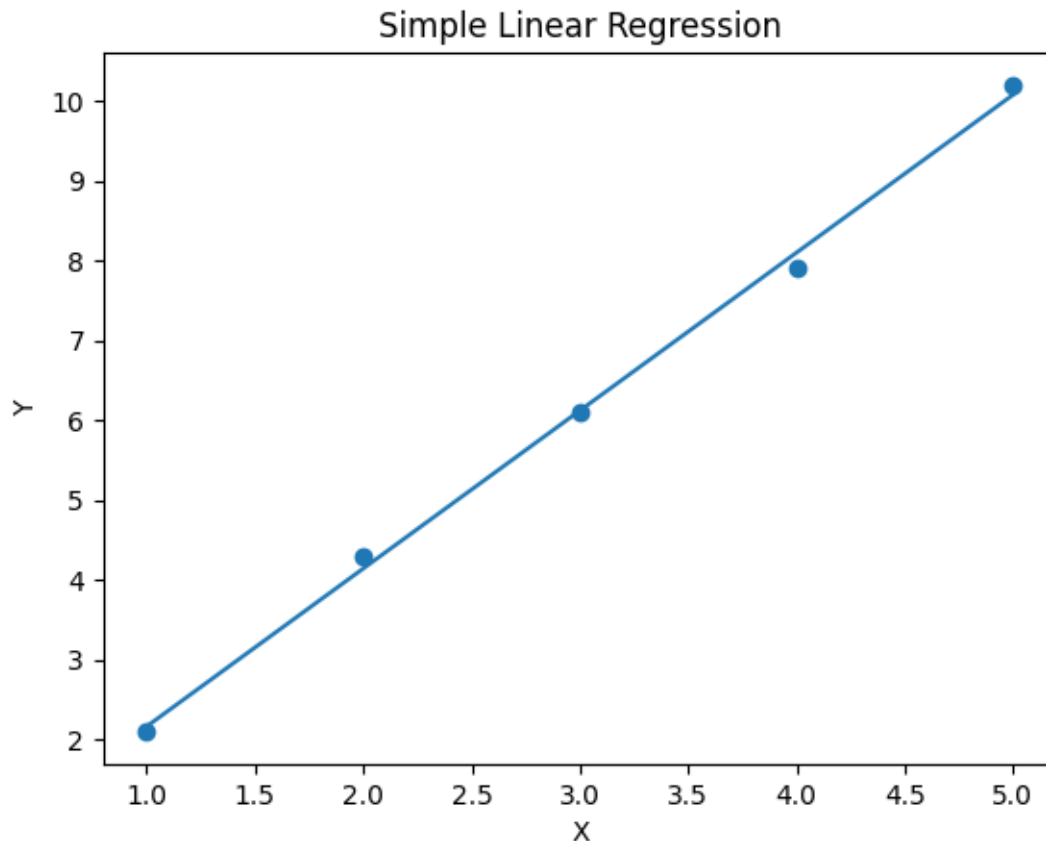
```
Y_pred = model.predict(X)
```

```
plt.scatter(X, Y)
```

```
plt.plot(X, Y_pred)
plt.xlabel("X")
plt.ylabel("Y")
plt.title("Simple Linear Regression")
plt.show()
```

```
print("Slope:", model.coef_[0])
print("Intercept:", model.intercept_)
```

output



```
Slope: 1.9800000000000004
Intercept: 0.179999999999999794
```

Question 7: Fit a **Multiple Linear Regression** model on this sample data:

- Area = [1200, 1500, 1800, 2000]
- Rooms = [2, 3, 3, 4]
- Price = [250000, 300000, 320000, 370000]

Check for multicollinearity using VIF and report the results.

Answer:

```

import pandas as pd
from sklearn.linear_model import LinearRegression
from statsmodels.stats.outliers_influence import variance_inflation_factor

data = pd.DataFrame({
    'Area':[1200,1500,1800,2000],
    'Rooms':[2,3,3,4],
    'Price':[250000,300000,320000,370000]
})

X = data[['Area','Rooms']]
y = data['Price']

model = LinearRegression()
model.fit(X,y)

# VIF
vif = pd.DataFrame()
vif["Feature"] = X.columns
vif["VIF"] = [variance_inflation_factor(X.values, i) for i in range(X.shape[1])]

print(vif)

```

Output

	Feature	VIF
0	Area	127.796923
1	Rooms	127.796923

Question 8: Implement **polynomial regression** on the following data:

- $X = [1, 2, 3, 4, 5]$
- $Y = [2.2, 4.8, 7.5, 11.2, 14.7]$

Fit a **2nd-degree polynomial** and plot the resulting curve.

Answer:

```

import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression

X = np.array([1,2,3,4,5]).reshape(-1,1)
Y = np.array([2.2,4.8,7.5,11.2,14.7])

```

```

poly = PolynomialFeatures(degree=2)
X_poly = poly.fit_transform(X)

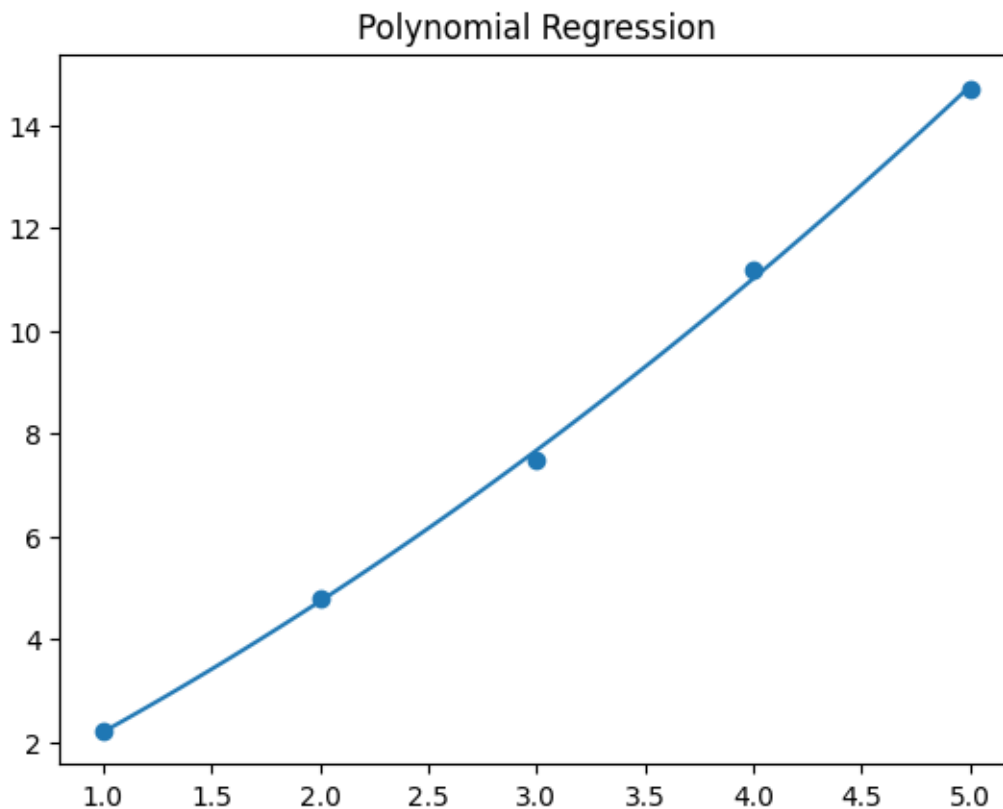
model = LinearRegression()
model.fit(X_poly, Y)

X_range = np.linspace(1,5,100).reshape(-1,1)
X_range_poly = poly.transform(X_range)

plt.scatter(X,Y)
plt.plot(X_range, model.predict(X_range_poly))
plt.title("Polynomial Regression")
plt.show()

```

Output



Question 9: Create a **residuals plot** for a regression model trained on this data:

- $X = [10, 20, 30, 40, 50]$
- $Y = [15, 35, 40, 50, 65]$

Assess heteroscedasticity by examining the spread of residuals.

(Include your Python code and output in the code box below.)

Answer:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression

X = np.array([1,2,3,4,5]).reshape(-1,1)
Y = np.array([2.2,4.8,7.5,11.2,14.7])

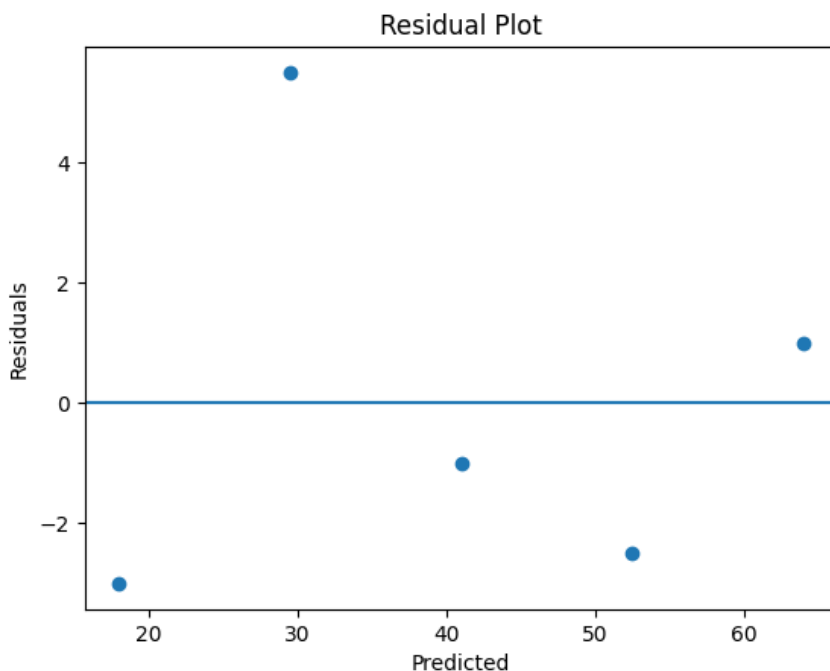
poly = PolynomialFeatures(degree=2)
X_poly = poly.fit_transform(X)

model = LinearRegression()
model.fit(X_poly, Y)

X_range = np.linspace(1,5,100).reshape(-1,1)
X_range_poly = poly.transform(X_range)

plt.scatter(X,Y)
plt.plot(X_range, model.predict(X_range_poly))
plt.title("Polynomial Regression")
plt.show()
```

Output



Question 10: Imagine you are a data scientist working for a real estate company. You need to predict house prices using features like area, number of rooms, and location. However, you detect **heteroscedasticity** and **multicollinearity** in your regression model. Explain the steps you would take to address these issues and ensure a robust model.

Answer:

To handle heteroscedasticity:

- **Apply log/sqrt transformation**
- **Use Weighted Least Squares**
- **Use robust standard errors**

To handle multicollinearity:

- **Remove highly correlated features**
- **Use PCA (Principal Component Analysis)**
- **Use Ridge/Lasso Regression**

Other steps:

- **Feature selection**
- **Cross-validation**
- **Regularization techniques**

